

EE2120: Electrical Measurements and Instrumentation

SCADA and IIoT Systems Integration with ESP32, MQTT and Ignition

Objective: Build a complete bidirectional SCADA/IIoT path. ESP32 nodes publish telemetry through Mosquitto. Ignition receives the data, stores history, generates alarms, displays Perspective dashboards, and sends control commands back to the ESP32.

Item	Exercise requirement
Devices	ESP01 and ESP02
MQTT device accounts	esp01, esp02
MQTT SCADA account	ignition_scada
MQTT test account	ee2120 — read/write
MQTT root	EE2120/
Broker	Mosquitto
SCADA	Ignition 8.1.48
HMI	Ignition Perspective
Required functions	Temperature, online/offline, LED control/status, alarm, historian

Scope: Students are expected to build the SCADA application and verify the complete data/control chain. The full laboratory setup guide is not provided as the student handout.

1. Exercise Objective

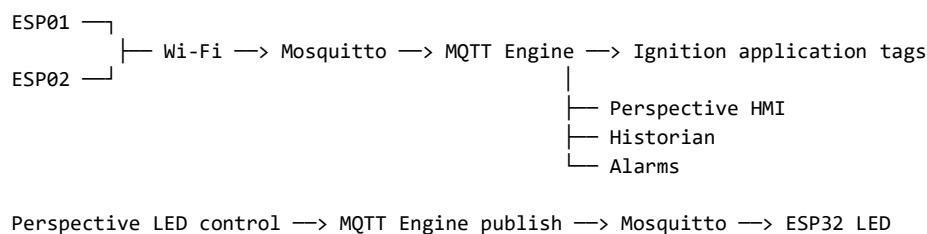
Build a two-device SCADA HMI in Ignition Perspective that monitors ESP01 and ESP02, controls their LEDs through MQTT, displays actual LED feedback, records temperature history, and generates high-temperature alarms.

1.1 What students are given

Provided item	Student use
Mosquitto .conf sample	Reference only — study the required broker settings
Mosquitto passwords sample	Reference only — study the intended MQTT accounts
Mosquitto ACL sample	Reference only — study authentication/authorization and topic permissions
Arduino ESP32 program	Use directly as the starting program; edit device-specific values and upload

Required: Do not treat the three Mosquitto files as files to blindly copy. Their purpose in this exercise is to let students understand how the working system is configured. The Arduino program is the file students directly edit, compile and upload.

1.2 Expected architecture



2. Prerequisites and Given Configuration

2.1 Software

- **Arduino IDE**
- **ESP32 board support by Espressif Systems**
- **PubSubClient Arduino library**
- **Mosquitto broker and command-line clients**
- **Ignition 8.1.48 or Later**
- **Cirrus Link MQTT Engine compatible with the installed Ignition version**
- **Ignition Perspective and Tag Historian**

2.2 Network

Parameter	Value
MQTT TCP port	1883
Ignition Gateway/Perspective HTTP port	8088
Broker address in ESP32	LAN IPv4 address of the Mosquitto/Ignition PC
Broker address in MQTT Engine	tcp://localhost:1883 when Mosquitto and Ignition are on the same PC

Critical: Do not use localhost or 127.0.0.1 as `mqtt_server` in the ESP32. From the ESP32, localhost refers to the ESP32 itself. This setup guide explicitly uses the server's LAN IPv4 address for the ESP32.

3. MQTT Users, Topics and Permissions

3.1 Required users

Username	Role in exercise
esp01	ESP01 device identity
esp02	ESP02 device identity
ignition_scada	MQTT Engine service identity
ee2120	Read/write laboratory testing identity

3.2 Topic structure

Function	ESP01	ESP02
Temperature	EE2120/ESP01/temp	EE2120/ESP02/temp
LED command	EE2120/ESP01/LED/cmd	EE2120/ESP02/LED/cmd
LED status	EE2120/ESP01/LED/status	EE2120/ESP02/LED/status
Device status	EE2120/ESP01/status	EE2120/ESP02/status

Critical: The temperature and LED command paths are directly established by the supplied working guide. LED/status and device-status topics are required by this exercise because the HMI must display LED feedback and online/offline state. Their exact implementation must match the Arduino program supplied for the exercise; do not invent a different topic name in one layer.

3.3 Intended permissions

User	Own telemetry/status	Own LED command	Other ESP topics
esp01	Publish	Subscribe	Denied
esp02	Publish	Subscribe	Denied
ignition_scada	Read	Write	Only the required namespace
ee2120	Read/write for testing	Read/write for testing	Read/write for testing

Source boundary: The supplied reference ACL in the original setup explicitly gives ESP users write access to their own temperature and read access to their own LED command topic; it gives ignition_scada read access to temperature and write access to LED command, and ee2120 read/write access to EE2120/#. If the supplied reference ACL differs for LED/status or device status, use the exercise's actual supplied configuration as the authority.

4. Prepare and Program the ESP32s

4.1 Arduino IDE

1. Connect an ESP32 with a USB data cable.
2. Open Arduino IDE.
3. Install the ESP32 board package if it is not already installed.
4. Select the appropriate ESP32 board.
5. Select the correct COM port.
6. Install PubSubClient by Nick O'Leary.
7. Open the supplied Arduino program.

4.2 Configure ESP01

8. Enter the laboratory Wi-Fi SSID and password.
9. Enter the Mosquitto/Ignition PC LAN IP address as mqtt_server.
10. Set mqtt_port to 1883.
11. Set the MQTT username to esp01.
12. Enter the password assigned to esp01.
13. Set the device identifier to ESP01.
14. Verify the temperature topic is EE2120/ESP01/temp.
15. Verify the LED command topic is EE2120/ESP01/LED/cmd.
16. Verify the LED status topic and device-status topic match the exercise topic table.
17. Compile and upload the program.
18. Open Serial Monitor at 115200 baud.
19. Verify Wi-Fi connection, MQTT connection, command subscription and temperature publication.

4.3 Configure ESP02

20. Use the same supplied Arduino program.
21. Change the MQTT username to esp02.
22. Enter the password assigned to esp02.
23. Change the device identifier to ESP02.
24. Change every device-specific MQTT topic from ESP01 to ESP02.
25. Compile and upload.
26. Verify Wi-Fi and MQTT connection.
27. Verify temperature publication and LED command reception.

Reference: The supplied working program publishes a generated temperature once per second and uses the ESP32 built-in LED for the control exercise. If the supplied student program contains the status/LED-feedback logic, preserve it; only change the device-specific configuration required for ESP02.

5. Verify MQTT Before Opening Ignition

5.1 Temperature

28. Use the ee2120 test account to subscribe to ESP01 temperature.
29. Verify values arrive on EE2120/ESP01/temp.
30. Repeat for ESP02.

```
cd "C:\Program Files\Mosquitto"
```

```
mosquitto_sub -d -h 127.0.0.1 -p 1883 -i PC_TEST -u ee2120 -P "YOUR_EE2120_PASSWORD" -t "EE2120/ESP01/temp" -v
```

5.2 LED command

```
mosquitto_pub -h 127.0.0.1 -p 1883 -u ee2120 -P "YOUR_EE2120_PASSWORD" -t "EE2120/ESP01/LED/cmd" -m "1"
```

```
mosquitto_pub -h 127.0.0.1 -p 1883 -u ee2120 -P "YOUR_EE2120_PASSWORD" -t "EE2120/ESP01/LED/cmd" -m "0"
```

31. Confirm the ESP01 LED changes state.
32. Repeat for ESP02.
33. If LED status feedback is implemented, verify the status topic changes after the command.

5.3 Verify device isolation

ESP01 own telemetry – expected to be permitted

```
mosquitto_pub -d -h 127.0.0.1 -p 1883 -u esp01 -P "YOUR_ESP01_PASSWORD" -t "EE2120/ESP01/temp" -m "99"
```

ESP01 writing ESP02 telemetry – expected to be rejected

```
mosquitto_pub -d -h 127.0.0.1 -p 1883 -u esp01 -P "YOUR_ESP01_PASSWORD" -t "EE2120/ESP02/temp" -m "99"
```

Concept: This test demonstrates the difference between MQTT authentication and authorization: the username proves identity; the ACL determines which topics that identity may access.

6. Configure MQTT Engine in Ignition

6.1 Install/verify MQTT Engine

34. Open the Ignition Gateway.
35. Confirm the Gateway is running and the version is 8.1.48.
36. Open Config → System → Modules.
37. Verify/install the compatible Cirrus Link MQTT Engine module.
38. Confirm MQTT Engine is Running.

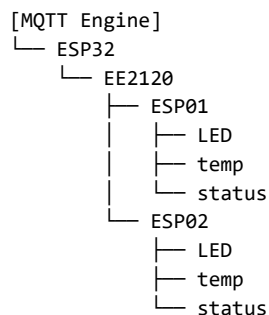
6.2 Connect MQTT Engine to Mosquitto

39. Open Gateway → Config → MQTT Engine → Settings.
40. Open MQTT Servers.
41. Create or edit the server named Mosquitto.
42. Set URL to tcp://localhost:1883.
43. Set Server Set to Default Set.
44. Set Username to ignition_scada.
45. Enter the ignition_scada password.
46. Save.
47. Confirm Status = Connected.

6.3 Configure the custom namespace

48. Open Gateway → Config → MQTT Engine → Namespaces → Custom.
49. Create or edit the namespace.
50. Set Name to ESP32.
51. Set Subscription to EE2120/#.
52. Set Root Tag Folder to ESP32.
53. Use the simple/non-JSON payload setting required by the supplied configuration.
54. Save.
55. Keep both ESPs publishing while checking discovery.

Expected path:



Source-derived: The supplied setup guide documents the namespace name ESP32, subscription EE2120/# and root folder ESP32. It also shows MQTT Engine as the communications layer and [default] tags as the SCADA application layer.

7. Create the SCADA Application-Layer Tags

Critical: Do not place the alarm and historian directly on MQTT Engine-managed tags. The working setup explicitly moved the alarm/history to a normal [default] reference tag because the direct MQTT Engine tag approach did not behave correctly.

7.1 Temperature tags

56. In Designer, select the default provider.
57. Create a Reference Tag for ESP01 temperature.

58. Point it to [MQTT Engine]ESP32/EE2120/ESP01/temp.
59. Create the equivalent Reference Tag for ESP02.
60. Confirm both application tags follow their MQTT Engine source.

Application tag	Source
[default]ESP01	[MQTT Engine]ESP32/EE2120/ESP01/temp
[default]ESP02	[MQTT Engine]ESP32/EE2120/ESP02/temp

7.2 Online/offline tags

61. Create a Boolean application tag for ESP01 status.
62. Create the equivalent tag for ESP02.
63. Connect each to the exercise's device-status mechanism.
64. Define TRUE as ONLINE and FALSE as OFFLINE.
65. Disconnect an ESP and verify the corresponding state changes.

Application tag	Required meaning
[default]esp01_status	TRUE = online; FALSE = offline
[default]esp02_status	TRUE = online; FALSE = offline

Engineering distinction: Do not call a status tag 'Wi-Fi status' unless it actually measures Wi-Fi association. An end-to-end ESP status is a communication-health indication.

7.3 LED status tags

66. Create an application-layer LED status tag for ESP01.
67. Create the equivalent tag for ESP02.
68. Use the datatype and state representation required by the supplied ESP/status implementation.
69. If using the established String implementation, represent the states as UNKNOWN, 1 and 0.
70. Use a value-change event on the MQTT Engine LED/status source where required.
71. Write the application value only when the incoming MQTT value has good quality.

Example valueChanged event pattern:

```
def valueChanged(tag, tagPath, previousValue, currentValue, initialChange, missedEvents):
    if currentValue.quality.isGood():
        system.tag.writeBlocking(
            ["[default]ESP LED Status Tags/ESP01_LED_Status"],
            [currentValue.value]
        )
```

Critical: Change only the destination tag path for ESP02. The source MQTT Engine tag path must be the actual tag generated by the configured namespace.

8. Configure the Historian

8.1 Temperature history

72. Confirm the Tag Historian module is Running.
73. Confirm the configured history provider is available.
74. Enable history on the ESP01 application temperature tag.
75. Select the configured storage provider.
76. Enable history on the ESP02 application temperature tag.
77. Allow data to collect for at least 30–60 seconds.
78. Verify historical records exist.

Source-derived: The completed setup used a storage provider named `test_tag_hist`. If the lab Gateway already has the configured provider, use that provider rather than creating a second one.

8.2 Perspective Time Series Chart

79. Add a Time Series Chart to the Perspective view.
80. Bind `props.series` to Tag History.
81. For the first test, use Direct tag selection.
82. Select the two application temperature tags.
83. Set Return Format = Wide.
84. Set Query Mode = PointCount.
85. Set Point Count = 100.
86. Set Time Range = Realtime.
87. Set Most Recent = 5 MINUTE.
88. Enable polling.
89. Set polling to 1 second.
90. Apply and save.
91. Verify that both temperature traces appear and update.

Critical: Start with direct Tag History selection. Do not use `tag()` as a substitute for a historian query: `tag()` returns a current value; the chart's Tag History binding retrieves historical data.

9. Configure High-Temperature Alarms

92. Edit the ESP01 application temperature tag.
93. Open Alarms.
94. Add an alarm named `High_Temperature_ESP01`.
95. Set Enabled = true.
96. Set Mode = Above Setpoint.
97. Set the required temperature setpoint.
98. Set Priority = High.
99. Use Active Delay = 0 for the initial exercise test unless instructed otherwise.
100. Save.
101. Repeat for ESP02 with a distinct alarm name.
102. Raise each temperature above its threshold and verify activation.

Source-derived: The supplied working guide documents a `High_Temperature` alarm on the normal [default] Temperature tag and explicitly notes that placing it directly on the MQTT Engine-managed tag did not work correctly. For this two-device exercise, use the corresponding normal application temperature tags.

9.1 Alarm Status Table

103. Add an Alarm Status Table to the Perspective view.
104. Use the default active-alarm source unless a filter is required.
105. Save the project.
106. Trigger ESP01's alarm.

107. Confirm the alarm appears.
108. Repeat for ESP02.
109. Verify the alarm is active before troubleshooting the table if it does not appear.

10. Build the Perspective HMI

10.1 Layout

- ESP01 monitoring section
- ESP02 monitoring section
- LED controls
- LED status indicators
- Alarm Status Table
- Temperature Time Series Chart

10.2 Temperature labels

110. Create an ESP01 temperature label.
111. Bind it to [default]ESP01.
112. Create an ESP02 temperature label.
113. Bind it to [default]ESP02.
114. Format the displayed values appropriately.

10.3 Online/offline labels

115. Create an ESP01 status label.
116. Bind it to esp01_status.
117. Display ONLINE when TRUE.
118. Display OFFLINE when FALSE.
119. Repeat for ESP02.
120. Add a visual indicator such as a green/red dot if required.

Example expression for ESP01:

```
if({[default]esp01_status}, "ONLINE", "OFFLINE")
```

Example indicator:

```
if({[default]esp01_status}, "#0EF303", "#EF4444")
```

10.4 LED status labels

121. Create an ESP01 LED status label.
122. Bind it to the ESP01 application LED status tag.
123. Display ON, OFF or UNKNOWN.
124. Repeat for ESP02.
125. Use a clear visual distinction between ON, OFF and UNKNOWN.

If the application LED status is a String containing "1" or "0":

```
switch(  
    tag("[default]ESP LED Status Tags/ESP01_LED_Status"),  
    "1", "ON",  
    "0", "OFF",  
    "UNKNOWN"  
)
```

Datatype: If your actual LED application tag is Boolean rather than String, do not use the String comparison above. Match the expression to the actual datatype.

11. Implement LED ON/OFF Controls

11.1 ESP01 buttons

```
def runAction(self, event):
    system.cirruslink.engine.publish(
        "Mosquitto",
        "EE2120/ESP01/LED/cmd",
        "1".encode(),
        0,
        False
    )

def runAction(self, event):
    system.cirruslink.engine.publish(
        "Mosquitto",
        "EE2120/ESP01/LED/cmd",
        "0".encode(),
        0,
        False
    )
```

11.2 ESP02 buttons

- 126. Duplicate the ESP01 ON/OFF controls.
- 127. Change the topic from ESP01 to ESP02.
- 128. Verify the MQTT Engine server name is exactly Mosquitto.
- 129. Save and test each button.

11.3 Verify feedback rather than command alone

- 130. Press LED ON.
- 131. Verify the physical LED turns ON.
- 132. Verify the ESP publishes LED status feedback.
- 133. Verify MQTT Engine receives the status.
- 134. Verify the application LED status tag changes.
- 135. Verify the HMI label changes to ON.
- 136. Repeat for OFF and for the second ESP.

SCADA principle: The HMI should display the actual feedback state, not simply change its label because the button was pressed. This separates a command/request from the confirmed device state.

12. Complete Functional Test

- ☐ ESP01 publishes temperature.
- ☐ ESP02 publishes temperature.
- ☐ ESP01 is shown ONLINE.
- ☐ ESP02 is shown ONLINE.
- ☐ ESP01 LED ON command works.
- ☐ ESP01 LED OFF command works.
- ☐ ESP02 LED ON command works.
- ☐ ESP02 LED OFF command works.
- ☐ ESP01 LED status feedback is displayed.
- ☐ ESP02 LED status feedback is displayed.
- ☐ ESP01 temperature history is recorded.
- ☐ ESP02 temperature history is recorded.

- ☐ Both temperatures appear on the Time Series Chart.
- ☐ ESP01 high-temperature alarm activates.
- ☐ ESP02 high-temperature alarm activates.
- ☐ Alarm Status Table displays the active alarm.
- ☐ Disconnecting ESP01 changes its status to OFFLINE.
- ☐ Reconnecting ESP01 restores ONLINE state.
- ☐ Disconnecting ESP02 changes its status to OFFLINE.
- ☐ Reconnecting ESP02 restores ONLINE state.

13. Troubleshooting Order

Method: Do not jump directly to Perspective when a value is missing. Trace the signal from the physical device toward the HMI.

Symptom	Check in this order
No temperature in HMI	ESP serial output → MQTT topic → MQTT Engine tag → application tag → Perspective binding
MQTT Engine tag missing	ESP publication → broker → EE2120/# namespace → exact topic path
LED button does nothing	Perspective script → MQTT Engine server name → broker → ESP subscription → physical LED
LED changes but label does not	ESP LED/status publication → ACL → MQTT Engine status tag → application tag → HMI binding
Online/offline wrong	ESP status source → application status tag → HMI binding
Historian empty	History Enabled → provider → stored records → Tag History binding
Alarm not visible	Alarm IsActive → alarm configuration → Alarm Status Table
Remote Perspective unavailable	server IP → ping → TCP 8088 → firewall → Gateway → Perspective session

13.1 Common errors

- Using localhost as the ESP32 MQTT broker address.
- Using the wrong MQTT username/password.
- Changing a topic in the Arduino program without changing the ACL.
- Changing the topic without checking the MQTT Engine namespace.
- Putting alarms/history directly on the MQTT Engine-managed tag instead of the application tag.
- Using tag() when a historical Tag History binding is required.
- Comparing a String LED tag with numeric 0/1 instead of "0"/"1".
- Changing the HMI state immediately on a button press instead of waiting for device feedback.
- Running two Mosquitto broker instances on port 1883.

14. Required Demonstration

137. Show the MQTT Engine tags for ESP01 and ESP02.
138. Show both live temperature values in Perspective.
139. Show both online/offline states.
140. Turn ESP01 LED ON and OFF from Perspective.
141. Turn ESP02 LED ON and OFF from Perspective.
142. Show the LED status labels changing from actual feedback.
143. Trigger a high-temperature alarm.
144. Show the alarm in the Alarm Status Table.
145. Show historical temperature data in the Time Series Chart.
146. Disconnect one ESP and demonstrate the offline indication.
147. Reconnect the ESP and demonstrate recovery.